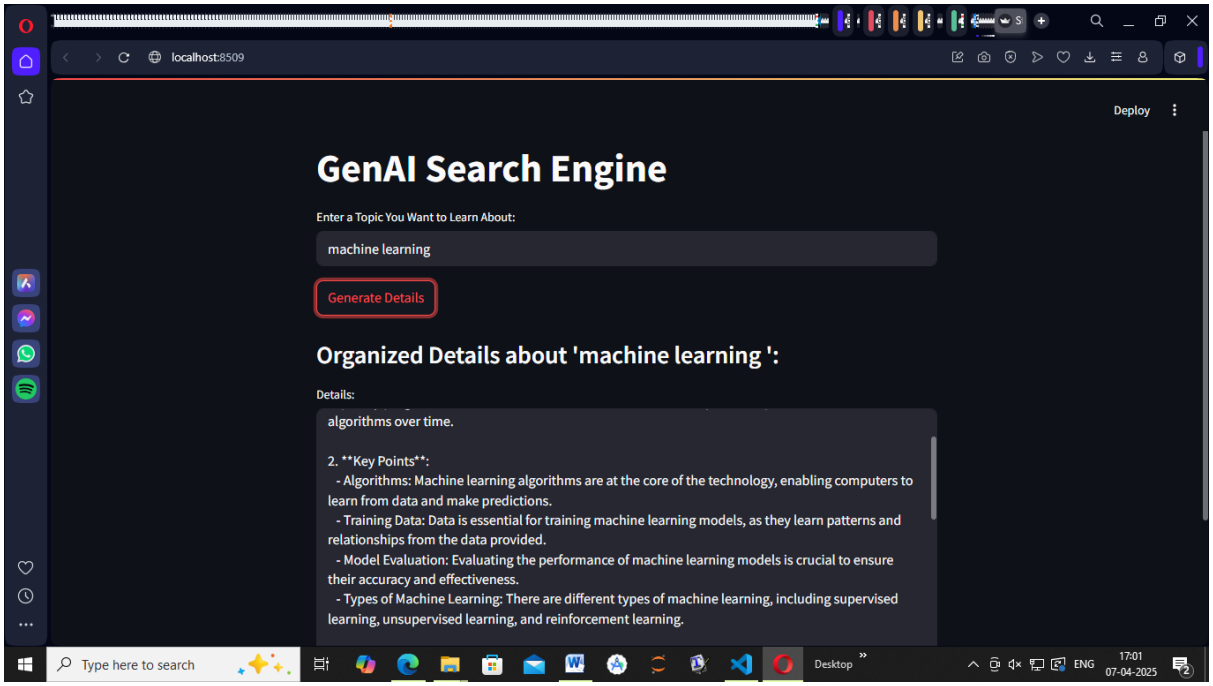
```python
import openai
import streamlit as st

# Set your OpenAI API key
openai.api_key = 'sk-proj-W43ulaTle3anKOH3h7WwRpMcyqfvQ_dWRT4ss95L8J-
BN3XzEwOaytoO4C1LdfTB07gmz_4EihT3BlbkFJGPDARtMX6Djt5nCGxeUBLp9OJ5d_qcBkuE3DBRu
3a-J4Flt1Qt37sa8oFH-yyCbx4d1hLSCwUA'  # Replace this with your OpenAI API key

# Streamlit UI setup
st.title("GenAI Search Engine")

# Input field for the user to enter a topic
topic = st.text_input("Enter a Topic You Want to Learn About:")

# Button to trigger the search and generate organized details
if st.button("Generate Details"):
    if topic:
        # Constructing the prompt to ask the model for organized details
        prompt = f"Provide detailed and organized information about the topic
'{topic}'. Please format the information as follows:\n" \
                 f"1. **Introduction**: A brief overview of the topic.\n" \
                 f"2. **Key Points**: Highlight the key aspects or
components.\n" \
                 f"3. **Applications/Use Cases**: Explain how the topic is
used or applied in the real world.\n" \
                 f"4. **Challenges or Limitations**: Mention any issues or
limitations related to the topic.\n" \
                 f"5. **Conclusion**: Provide a brief conclusion about the
topic.\n"

        # Request the model to generate the response using gpt-3.5-turbo
        try:
            # Using `openai.ChatCompletion.create` for `gpt-3.5-turbo`
            response = openai.ChatCompletion.create(
                model="gpt-3.5-turbo",  # Updated model
                messages=[
                    {"role": "system", "content": "You are a helpful
assistant."},
                    {"role": "user", "content": prompt},
                ],
                max_tokens=500,  # Limit on the length of the response
                temperature=0.7,  # Controls randomness (0.7 is a good
balance)
                n=1,  # Generate 1 response
                stop=None  # No explicit stop sequence
            )

            # Extracting the generated text from the response
```

```
            organized_details =
response['choices'][0]['message']['content'].strip()

            # Display the organized details
            st.subheader(f"Organized Details about '{topic}':")
            st.text_area("Details:", organized_details, height=300)

        except Exception as e:
            st.error(f"Error: {str(e)}")

    else:
        st.error("Please enter a topic to search about.")
```